

Software Design



Introduction

Design phase transforms SRS document:

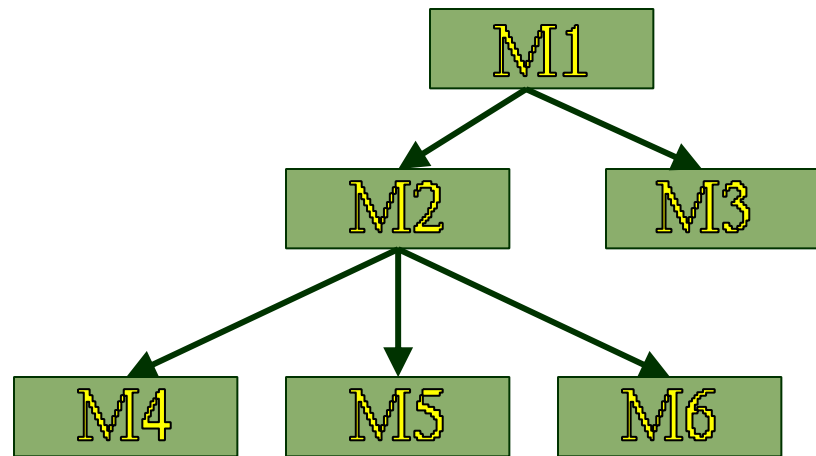
into a form easily implementable in some programming language.



Items Designed During Design Phase

1. module structure,
2. control relationship among the modules
3. interface among different modules,
data items exchanged among different modules,
4. data structures of individual modules,
5. algorithms for individual modules.

Module Structure



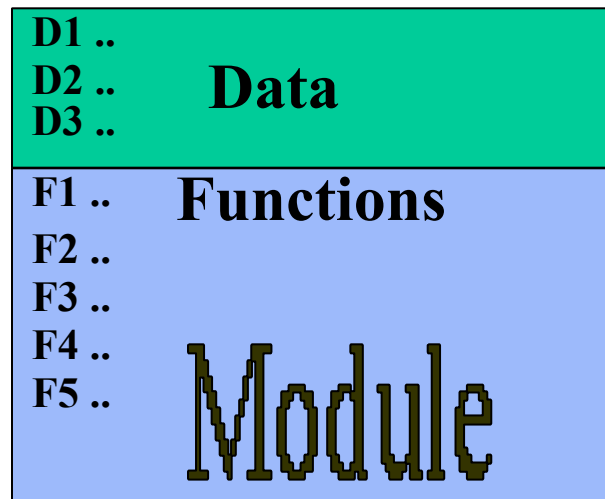
Example



Introduction

A module consists of:

1. several data members and functions
2. associated data structures.



Introduction

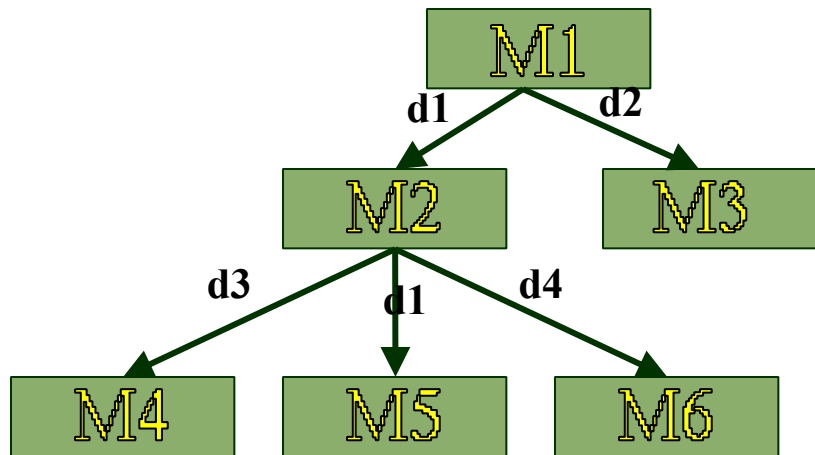
Design activities are usually classified into two stages:

1. preliminary (or high-level) design
2. detailed design.

High-level design

Identify:

1. modules
2. control relationships among modules
3. interfaces among modules.



What Is Good Software Design?

- Should identify all functionalities of the system correctly.
- Should be easily understandable.
- Should be efficient.
- Should be easy to change (i.e. easily maintainable)
- Should be Modular

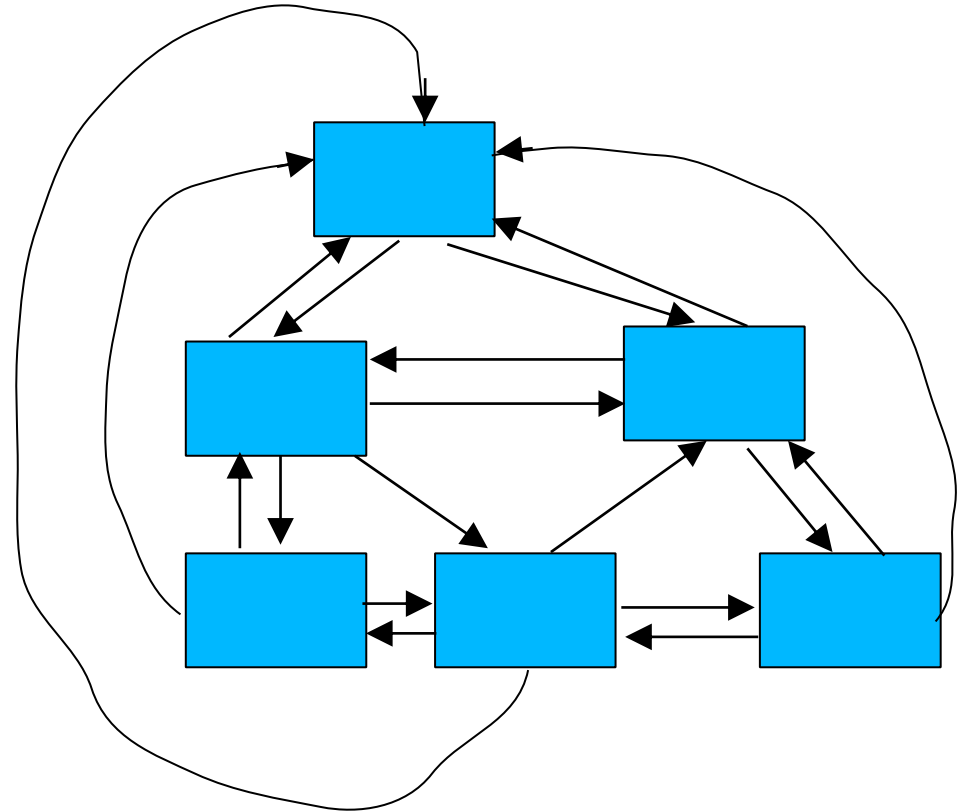
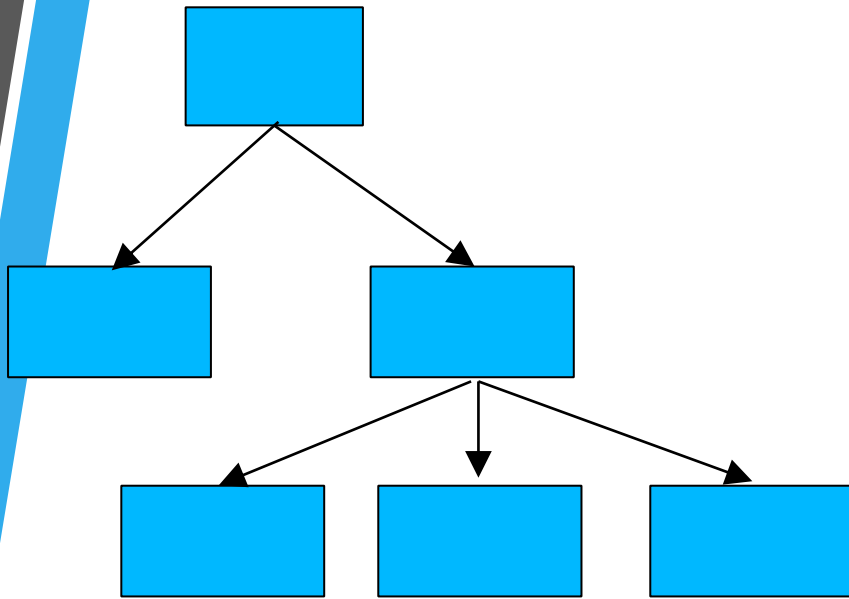
Modularity

Modularity is a fundamental attribute of any good design.

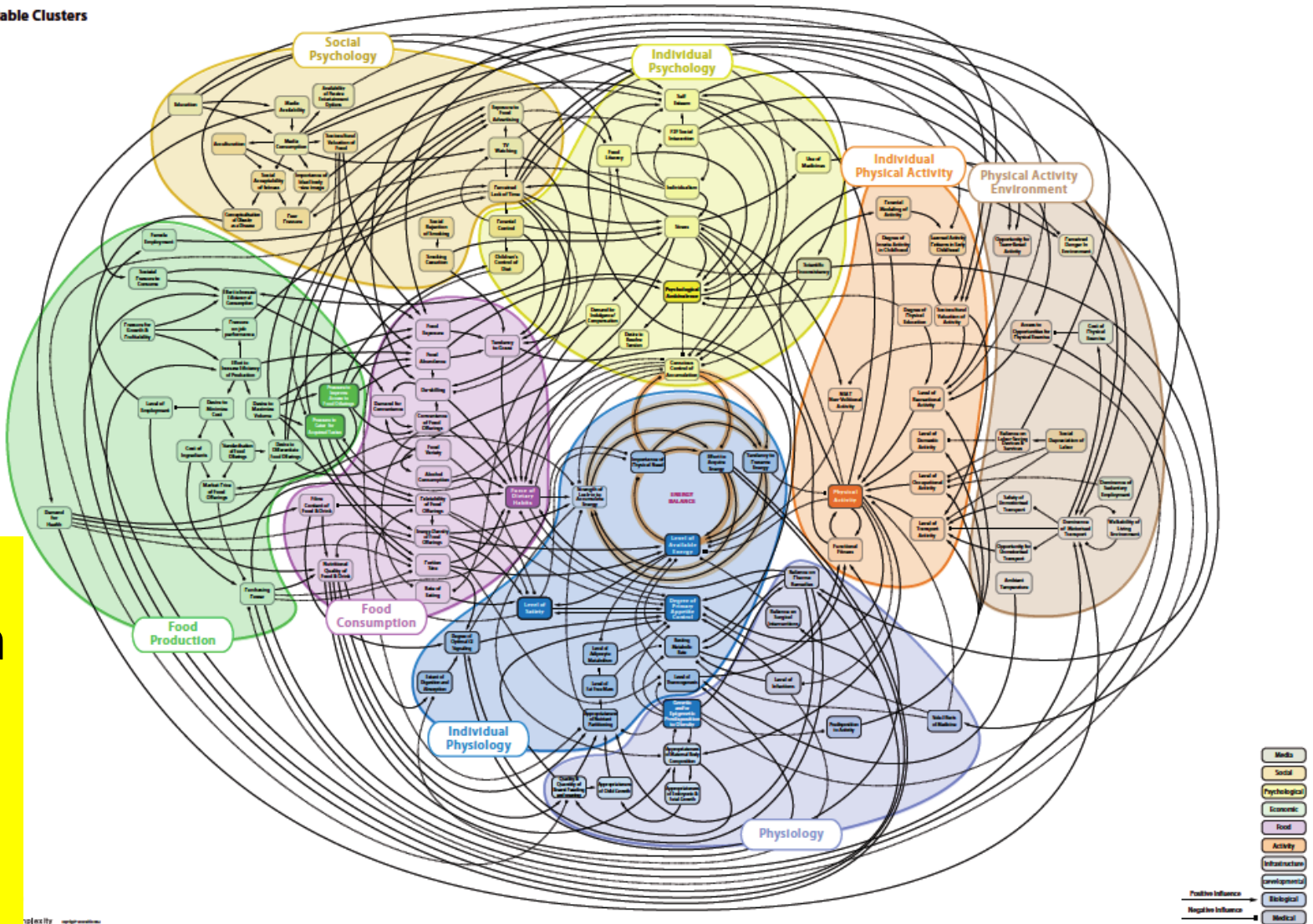
Decomposition of a problem cleanly into modules has some advantages.

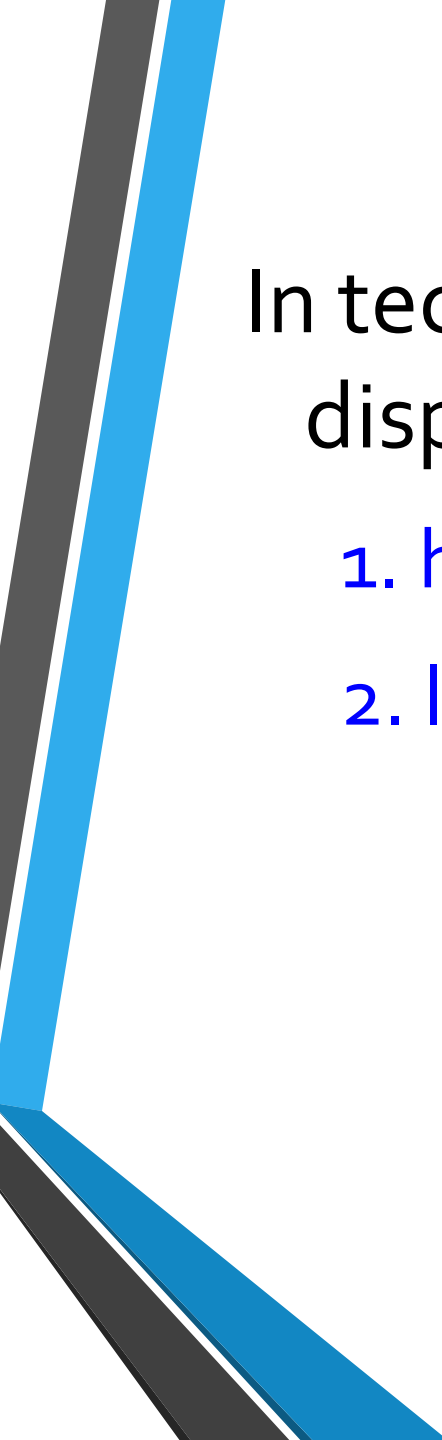
- modules can be understood clearly.
- reduces the complexity to great extent.

Example of Cleanly and Non-cleanly Decomposed Modules



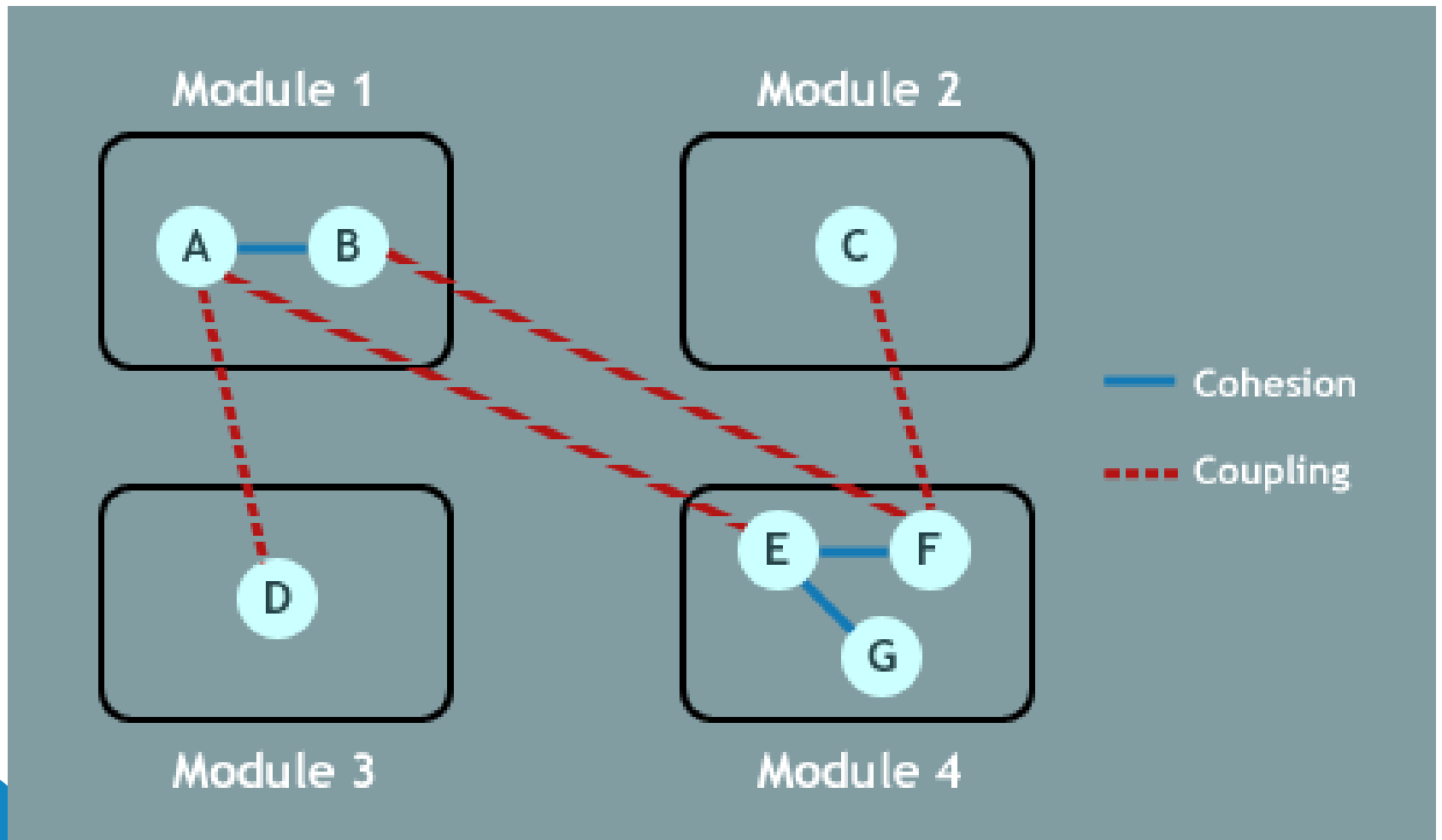
Variable Clusters

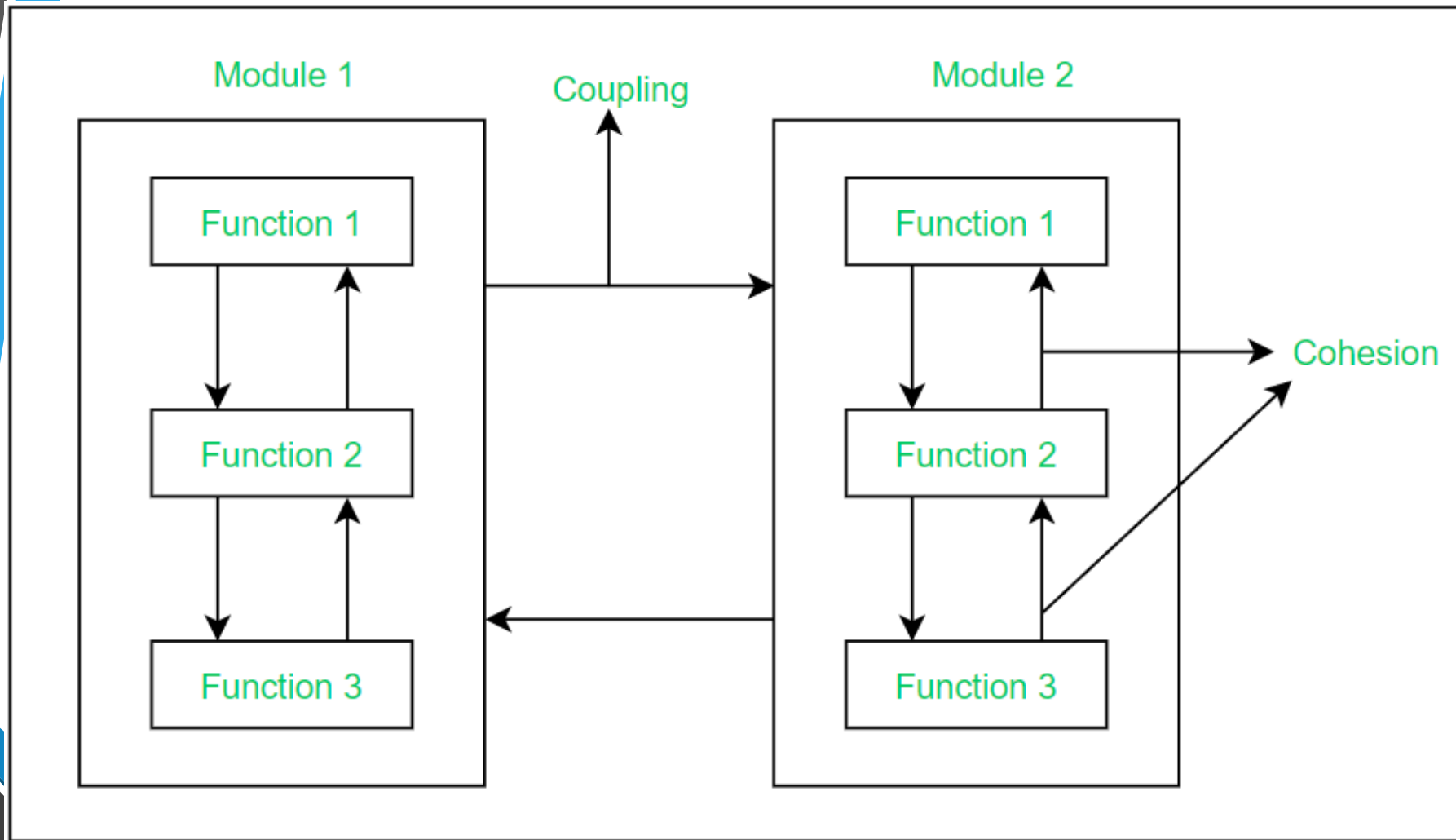




In technical terms, **modules** should display:

1. high cohesion
2. low coupling.





Cohesion vs Coupling

Cohesion	Coupling
While designing you should strive for high cohesion i.e. a cohesive component/ module focus on a single task (i.e., single-mindedness) with little interaction with other modules of the system.	While designing you should strive for low coupling i.e. Dependency between modules should be less.
Cohesion is Intra – Module Concept.	Coupling is Inter - Module Concept.

Cohesion and Coupling

Cohesion is a measure of:

- functional strength of a module.
- A cohesive module performs a single task or function.

Coupling between two modules:

- a measure of the degree of interdependence or interaction between the two modules.



Cohesion and Coupling

A module having high cohesion and low coupling:

A functionally independent module has minimal interaction with other modules.

Reuse: An Advantage of Functional Independence

A functionally independent module:

can be easily taken out and reused in a different program.

- each module does some well-defined and precise function
- the interfaces of a module with other modules is simple and minimal.

Advantages of Functional Independence

When degree of interaction between modules is low.

- Functional independence reduces error propagation.
 - an error existing in one module does not directly affect other modules.
 - Modules are independent so Reuse of modules is possible.
 - Better understandability and good design
 - Complexity of design is reduced
 - Different modules easily understood in isolation

Classification of Cohesiveness

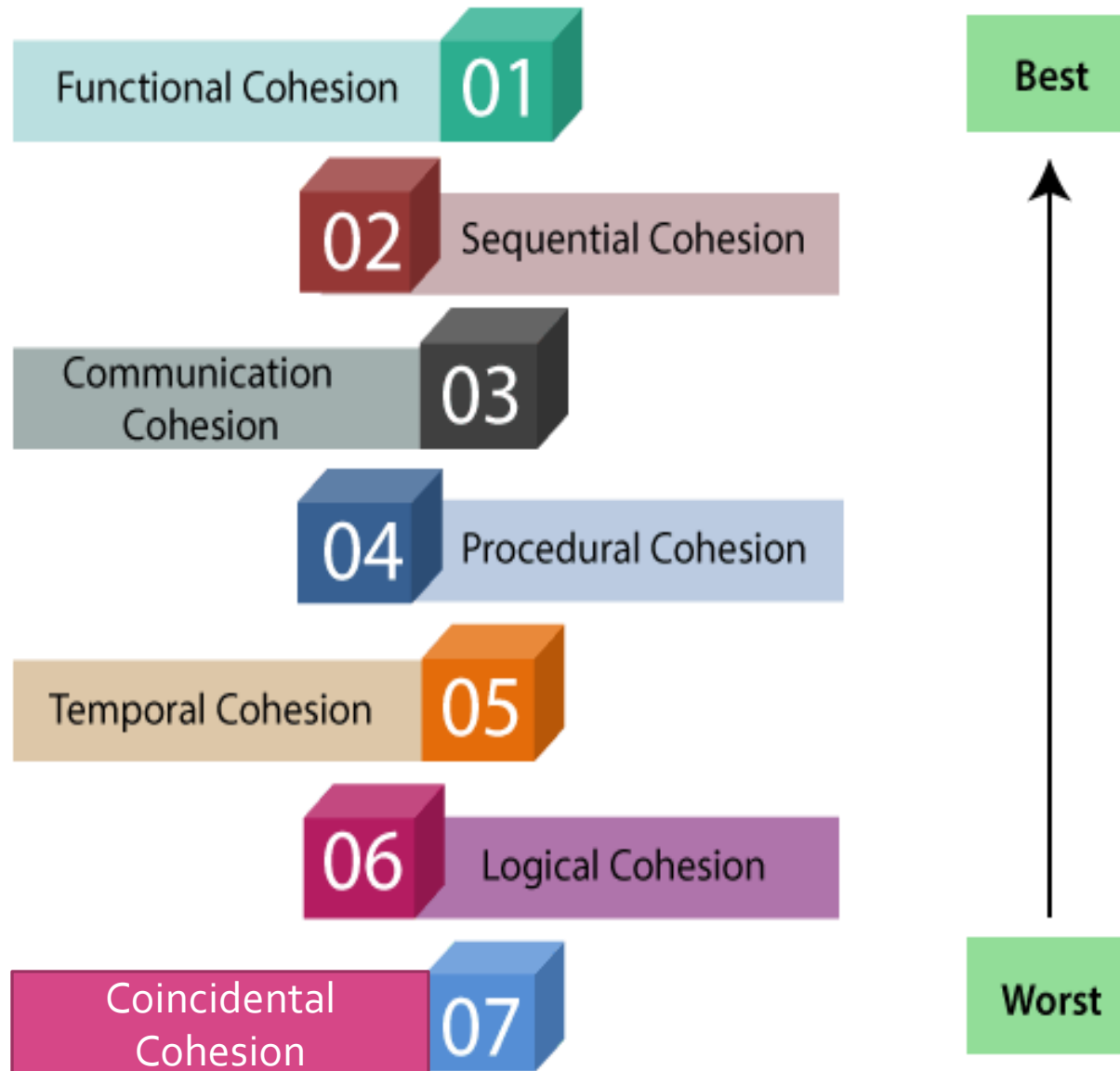


functional
sequential
communicational
procedural
temporal
logical
coincidental



**Degree of
cohesion**

Types of Modules Cohesion



Coincidental cohesion

The module performs a set of tasks:
which relate to each other very loosely, if at all.

- *the module contains a random collection of functions.*
- functions have been put in the module out of pure coincidence without any thought or design.

Coincidental Cohesion

- A module has coincidental cohesion if it performs multiple, completely unrelated actions
- Example:

```
print_next_line,  
reverse_string_of_second_parameter,  
add_7_to_fifth_parameter,  
convert_fourth_parameter_to_floating_point
```


Coincidental Cohesion - example

```
Module AAA{
```

```
    Print-inventory();
```

```
    Register-Student();
```

```
    Issue-Book();
```

```
};
```

Logical cohesion

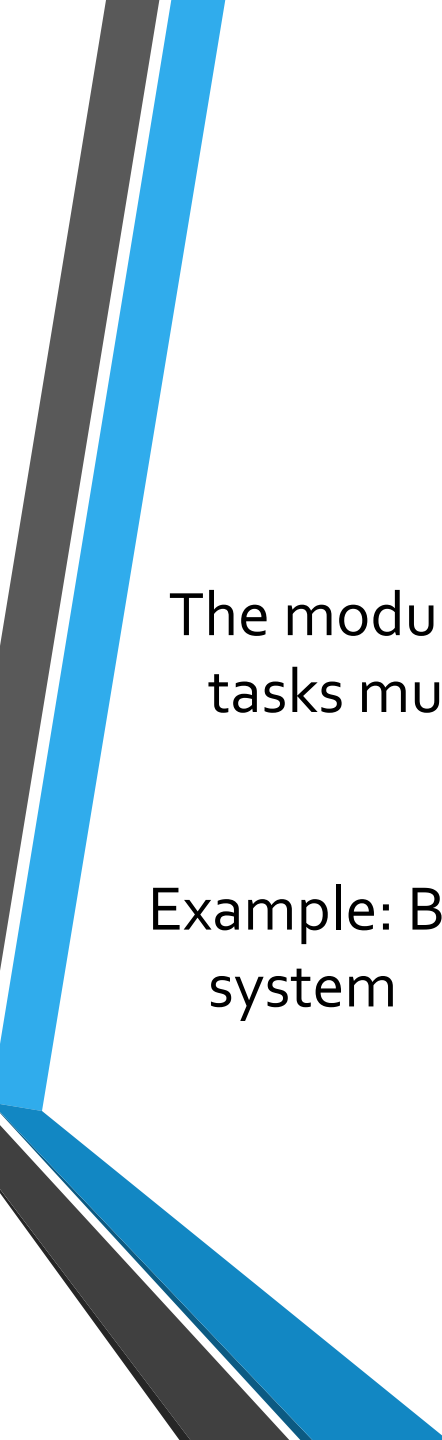
All elements of the module perform similar operations:

e.g. error handling, data input, data output, etc.

Example: a module containing set of print functions for generating reports such as grade sheet, salary slip, annual reports etc.

Logical Cohesion

```
module print{  
    void print-grades(student-file){ ...}  
  
    void print-certificates(student-file){...}  
  
    void print-salary(teacher-file){...}  
}
```



Temporal cohesion

The module contains tasks that are related by the fact that all the tasks must be executed in the same time span.

Example: Booting process of system, start-up and shut-down of system



```
init() {
```

```
    Check-memory();
```

```
    Check-Hard-disk();
```

```
    Initialize-Ports();
```

```
    Display-Login-Screen();
```

```
}
```

Temporal Cohesion - Example

Procedural cohesion

If the set of functions of the module all part of a procedure (algorithm) in which certain *sequence of steps have to be carried out in a certain order* for achieving an objective.

Procedural cohesion

e.g. 1. algorithm for decoding a message.

2. order processing module by sales clerk containing

login(),

check-order(),

print-bill(),

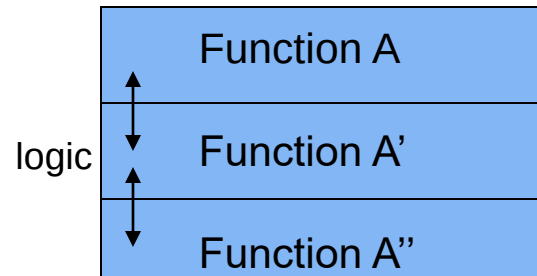
place-order-to-vendor(),

update-inventory().

Examples of Cohesion

Function A	
Function B	Function C
Function D	Function E

Coincidental
Parts unrelated



Logical
Similar functions

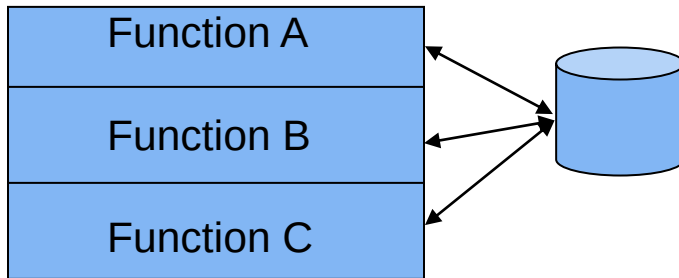
Time t_0
Time $t_0 + X$
Time $t_0 + 2X$

Temporal
Related by time

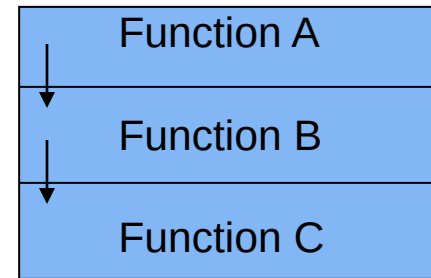
Function A
Function B
Function C

Procedural
Related by order of functions

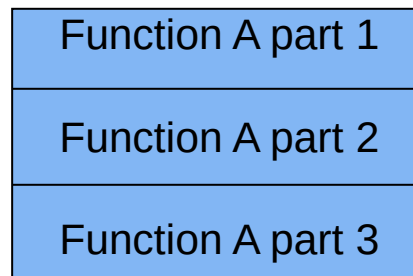
Examples of Cohesion (Cont.)



Communicational
Access same data



Sequential
Output of one is input to another



Functional
Sequential with complete, related functions

Communicational cohesion

If All functions of the module Refer to the same data structure,

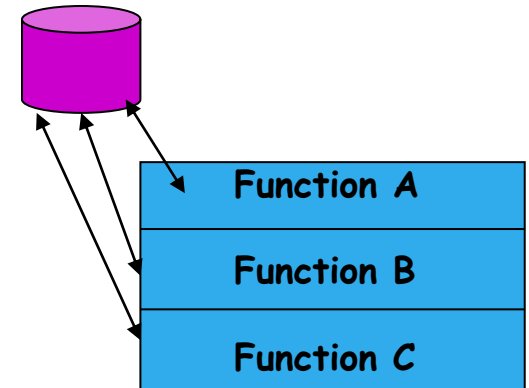
Example:

the set of functions defined on an array or a stack. Admit-students, enter-marks, printgradesheet etc

.

Communicational Cohesion

```
handle-Student- Data() {  
    Static Struct Student-data[10000];  
    Store-student-data();  
    Search-Student-data();  
    Print-all-students();  
};
```

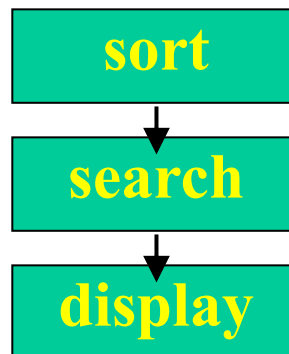


Communicational
Access same data

Sequential cohesion

If the Elements of a module forms different parts of a sequence, output from one element of the sequence is input to the next.

Example:



Functional cohesion

Different elements of a module cooperate:
to achieve a single function,
e.g. managing an employee's pay-roll.

When a module displays functional cohesion,

we can describe the function using a single sentence

Functional cohesion

If the Different elements of a module cooperate to achieve a single task.

Example: 1. a module containing all the functions required to manage employees's payroll shows functional cohesion

2. Module-name: managing-book-lending

Functions: issue-book, return-book, query-book

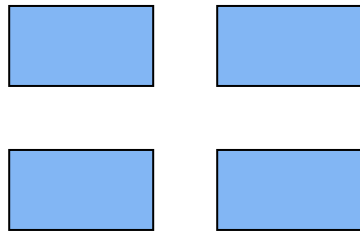
Coupling

Coupling indicates:

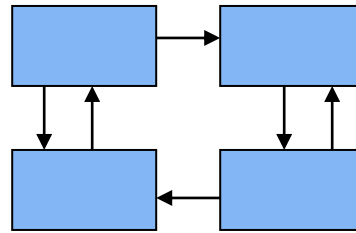
- how closely two modules interact or how interdependent they are.
- The degree of coupling between two modules depends on their interface complexity.

Coupling

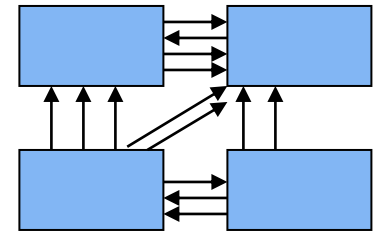
The degree of dependence such as the amount of interactions among components



No dependencies



Loosely coupled
some dependencies



Highly coupled
many dependencies

Classes of coupling

data
stamp
control
common
content

**Degree of
coupling**



Data coupling

Two modules are data coupled,

if they communicate by an elementary data item that is passed as a parameter between the two, eg an integer, a float, character etc.

Stamp coupling

Two modules are stamp coupled,

if they communicate via a composite data item:

➤ such as a record in PASCAL
or a structure in C.

Control coupling

It exists between two modules.

If Data from one module is used to direct order of instruction execution in another.

Example of control coupling:

a flag set in one module and tested in another module.



Common Coupling

Two modules are common coupled, if they share some global data items.

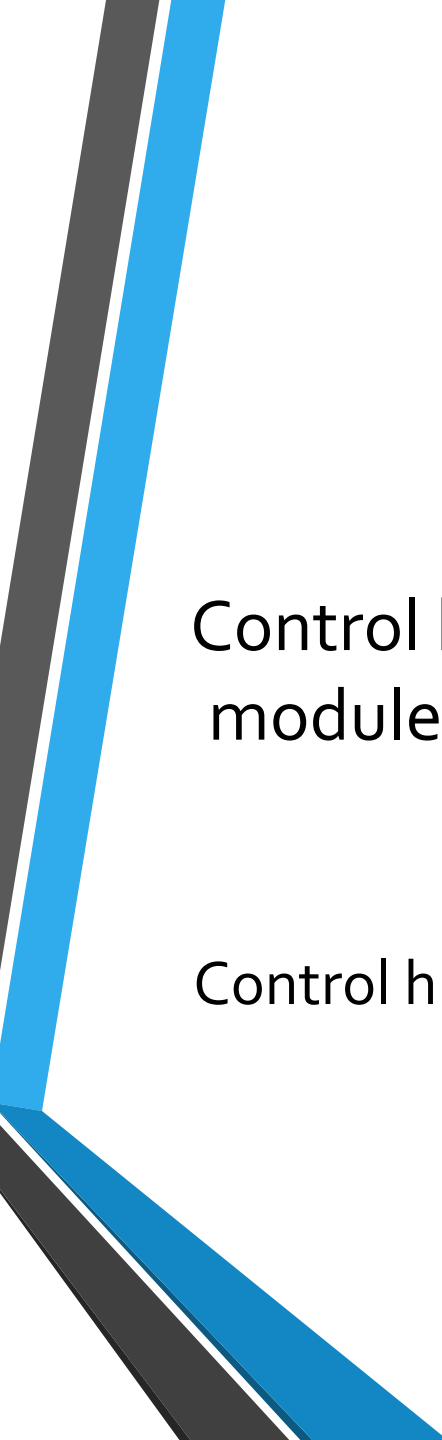
Content coupling

Content coupling exists between two modules:

if they share code,

e.g, branching from one module into another module.

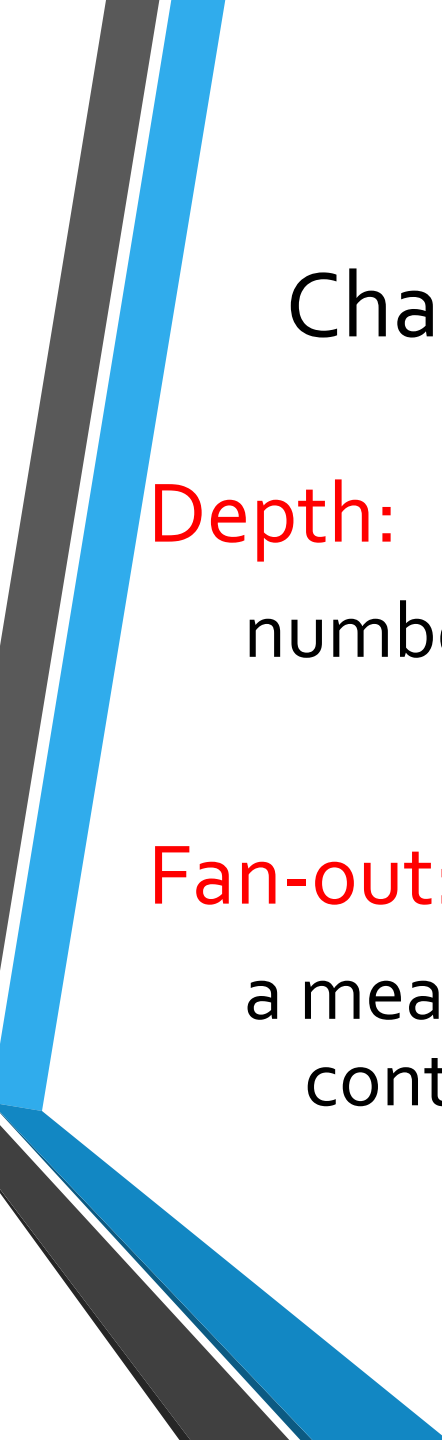
The degree of coupling increases from data coupling to content coupling.



Neat Hierarchy

Control hierarchy represents organization of modules.

Control hierarchy is also called program structure.



Characteristics of Module Structure

Depth:

number of levels of control

Fan-out:

a measure of the number of modules directly controlled by given module.

Characteristics of Module Structure

Fan-in:

indicates how many modules directly invoke a given module.

High fan-in represents code reuse and is generally encouraged.

Modularity

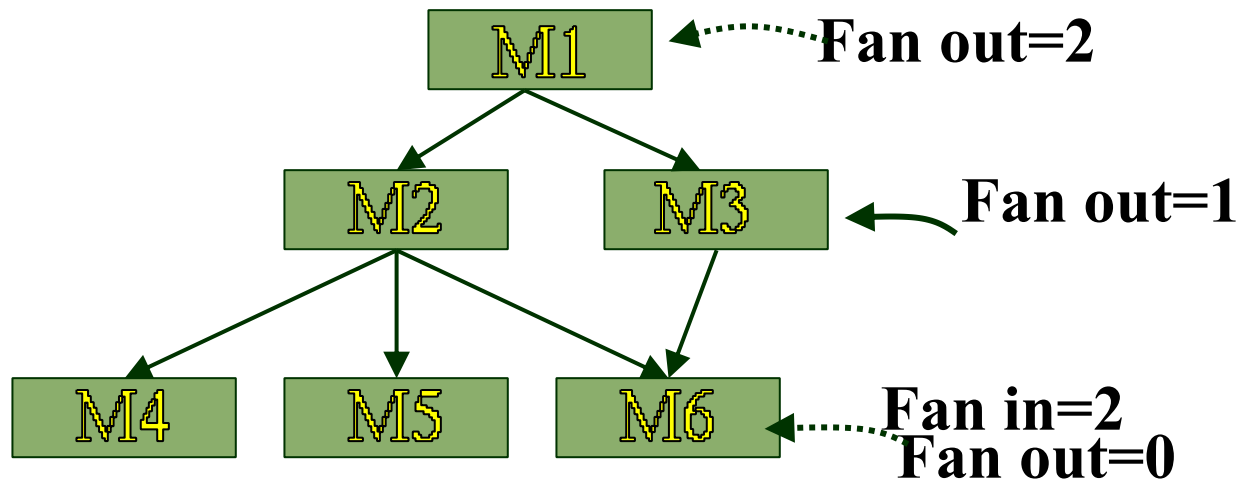
Neat arrangement of modules in a hierarchy means:

1. low fan-out

Fan-out: a measure of the number of modules directly controlled by given module.

2. abstraction

Module Structure



Goodness of Design

A design having modules:

with **high fan-out** numbers is not a good design:

a module having **high fan-out** lacks **cohesion**.

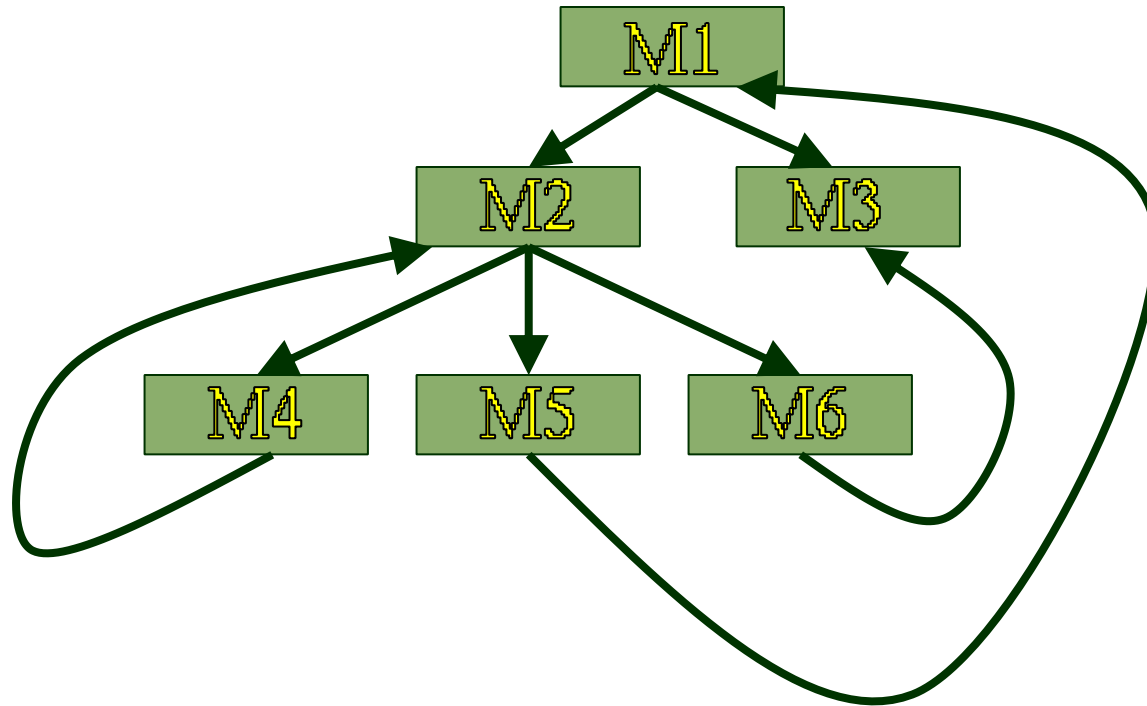
Visibility and Layering

A module A is said to be visible by another module B,

if A directly or indirectly calls B.

The layering principle requires modules at a layer can call only the modules immediately below it.

Bad Design



Abstraction

The principle of abstraction requires:

lower-level modules do not invoke
functions of higher level modules.

Also known as layered design.

25 Most Important features for e-commerce Website

